Exact SAT Solving and Constraint Programming for the N-Queens Problem

Pham Vinh Tri[a]

[a]*Faculty of Information Technology, VNU University of Engineering and Technology, 144 Xuan Thuy, Cau Giay Ward, Hanoi, 100000, Vietnam*

ABSTRACT

The N-Queens problem is a classic combinatorial optimization problem that requires finding a safe arrangement of queens on an $N \times N$ board. This report presents an exact approach based on Boolean Satisfiability (SAT) Solving. The main contribution is the mathematical modeling, implementation, and benchmarking of five At-Most-One (AMO) encodings: Binomial, Sequential Counter, Binary, Commander, and Product. Furthermore, we conduct a comprehensive cross-comparison against two other families of exact methods: Constraint Programming (OR-Tools CP-SAT, CPLEX CP) and Integer Linear Programming (IBM CPLEX MIP, Gurobi). Experimental results ranging from small instances ($N = 10$) up to very large search spaces ($N = 30$, with scalability studies at $N = 40$ and $N = 50$) reveal the severe trade-off between encoding size and solving time, and explain the root cause of the runtime bottleneck that SAT-based methods face on problems with global constraints. A cross-solver study (Glucose3/4, CaDiCaL, MapleSAT) further shows that the ranking between solvers reverses depending on the encoding, confirming that solver–encoding compatibility, not encoding shape alone, drives performance.

1. Introduction

Bài toán sắp xếp đồ thị (Graph layout) và phân bổ tài nguyên là một họ các bài toán tối ưu hóa tổ hợp có ứng dụng thực tiễn rộng rãi trong thiết kế VLSI, kiến trúc viễn thông và lập lịch [1]. Trong họ bài toán này, N-Queens nổi lên như một bài toán nền tảng mang tính chuẩn mực để thử nghiệm sức mạnh của các thuật toán tìm kiếm.

Mục tiêu của bài toán là sắp xếp N quân hậu trên bàn cờ sao cho khoảng cách tấn công theo luật cờ vua không bị vi phạm (không cùng hàng, cùng cột, hoặc cùng đường chéo). Khi chuyển dịch từ các bàn cờ nhỏ sang các bàn cờ kích thước lớn, sự bùng nổ tổ hợp (combinatorial explosion) khiến không gian trạng thái phình to theo hàm giai thừa. Các thuật toán quay lui (back-tracking) truyền thống nhanh chóng rơi vào trạng thái cạn kiệt tài nguyên tính toán.

Nghiên cứu này trình bày một cách tiếp cận SAT-based hoàn chỉnh nhằm giải quyết bài toán N-Queens một cách chính xác. Thông qua việc phân tích hành vi của bộ giải SAT (CDCL Heuristics) trước lượng lớn biến phụ, nghiên cứu phân tích định lượng nghịch lý về hiệu năng của các chuẩn mã hóa CNF. Phần còn lại của bài báo cáo được tổ chức như sau: Phần 2 định nghĩa bài toán. Phần 3 thiết lập các công thức mô hình hóa toán học cho 5 chuẩn mã hóa SAT cùng kỹ thuật phá đối xứng. Phần 4 mô tả mô hình Constraint Programming. Phần 5 trình bày thực nghiệm đối chuẩn giữa ba họ giải pháp (SAT, CP, ILP) và phân tích thắt cổ chai. Phần 6 là kết luận.

2. Related Work

Việc chuyển đổi các ràng buộc đếm (cardinality constraints) sang CNF là một chủ đề nghiên cứu trưởng thành trong lý thuyết SAT. Warners [2] đề xuất kỹ thuật mã hóa bất phương trình tuyến tính bằng biến phụ; Sinz [3] sau đó đưa ra chuẩn Sequential Counter với độ phức tạp tuyến tính $O(n)$ mệnh đề, trở thành nền tảng cho rất nhiều bộ giải hiện đại. Gent và Nightingale [4] phân tích sự đánh đổi giữa kích thước

ORCID(s):

mã hóa và hiệu quả lan truyền đơn vị. Cho ràng buộc AMO cụ thể, Klieber và Kwon [5] đề xuất chuẩn Commander, còn Chen [6] đề xuất chuẩn Product dựa trên phân rã ma trận 2D.

Về phía bộ giải, kiến trúc CDCL (Conflict-Driven Clause Learning) với heuristic VSIDS [7] là nền tảng của Glucose3 [8] — bộ giải được dùng trong thực nghiệm này. Song song đó, Constraint Programming [9] giữ nguyên cấu trúc ràng buộc cao cấp và xử lý `AllDifferent` bằng thuật toán luồng cực đại [10], tạo nên hướng tiếp cận đối chuẩn với SAT. Bên cạnh đó, Quy hoạch tuyến tính nguyên (Integer Linear Programming — ILP) [11] mô hình hóa bài toán bằng các bất phương trình khoảng và giải bằng Branch-and-Cut; nhờ branching trực tiếp trên biến gốc mà không sinh biến phụ, ILP thường tránh được sự nhiễu loạn heuristic của SAT. Nghiên cứu của chúng tôi đặt ba hướng này (SAT, CP, ILP) cạnh nhau trên cùng một bài toán, đồng thời làm rõ vai trò của cấu trúc mã hóa (chứ không chỉ kích thước) đối với hiệu năng bộ giải.

3. Problem Formulation

Bài toán N-Queens có thể được mô hình hóa dưới dạng một Bài toán Thỏa mãn Ràng buộc (Constraint Satisfaction Problem - CSP). Cho một bàn cờ kích thước $N \times N$, gọi $x_{i,j} \in \{0, 1\}$ là các biến nhị phân đại diện cho việc có đặt quân hậu tại tọa độ (i, j) hay không. Hệ thống ràng buộc của bài toán được mô tả như sau:

$$\sum_{j=1}^{N} x_{i,j} = 1 \quad \forall i \in \{1, \dots, N\} \tag{1}$$

$$\sum_{i=1}^{N} x_{i,j} \leq 1 \quad \forall j \in \{1, \dots, N\} \tag{2}$$

$$\sum_{i-j=k} x_{i,j} \leq 1 \quad \forall k \in \{-(N-1), \dots, N-1\} \tag{3}$$

$$\sum_{i+j=k} x_{i,j} \leq 1 \quad \forall k \in \{2, \dots, 2N\} \tag{4}$$

Phương trình (1) yêu cầu mỗi hàng có chính xác một quân hậu (Exactly-One). Các phương trình (2), (3), và (4) thuần túy là ràng buộc "Tối đa một" (At-Most-One - AMO). Ràng buộc Exactly-One có thể dễ dàng tách thành At-Least-One (ALO) và AMO. Do đó, nút thắt toán học của bài toán nằm hoàn toàn ở việc tối ưu hóa biểu diễn AMO.

4. SAT Encodings for At-Most-One Constraints

Để có thể áp dụng các bộ giải SAT hiện đại, các bất phương trình tuyến tính phải được chuyển đổi thành Dạng chuẩn hội (Conjunctive Normal Form - CNF). Cho một tập hợp các biến boolean $X = \{x_1, \dots, x_n\}$, ràng buộc AMO yêu cầu $\sum x_i \leq 1$. Dưới đây là phân tích toán học của 5 chuẩn mã hóa giải quyết yêu cầu này.

4.1. Binomial (Pairwise) Encoding

Phương pháp trực quan nhất là thiết lập mệnh đề loại trừ lẫn nhau trên mọi cặp biến. Với mọi $i \neq j$, không thể có trường hợp cả hai cùng mang giá trị True:

$$AMO_{Binomial}(X) = \bigwedge_{1 \leq i < j \leq n} (\neg x_i \vee \neg x_j) \tag{5}$$

Mã hóa này không sử dụng thêm bất kỳ biến phụ (auxiliary variable) nào, tuy nhiên nó tạo ra độ phức tạp không gian $O(n^2)$ mệnh đề.

4.2. Sequential Counter Encoding

Để giảm số lượng mệnh đề, Sinz (2005) [3] đề xuất sử dụng $n-1$ biến phụ $s_1, \dots, s_{n-1}$ đóng vai trò là một thanh ghi cộng dồn. Biến s_i mang giá trị True nếu tồn tại ít nhất một x_k ($k \leq i$) mang giá trị True.

$$(\neg x_1 \vee s_1) \tag{6}$$

$$\bigwedge_{i=2}^{n-1} ((\neg x_i \vee s_i) \wedge (\neg s_{i-1} \vee s_i) \wedge (\neg x_i \vee \neg s_{i-1})) \tag{7}$$

$$(\neg x_n \vee \neg s_{n-1}) \tag{8}$$

Chuẩn này đưa số mệnh đề về mức tuyến tính $O(n)$, đánh đổi bằng việc thêm vào hệ thống $O(n)$ biến phụ.

4.3. Binary, Commander và Product Encodings

Binary Encoding: Biểu diễn chỉ số của biến True (nếu có) thông qua hệ nhị phân bằng cách sử dụng $\lceil \log_2 n \rceil$ biến phụ, sinh ra $O(n \log n)$ mệnh đề.

Commander Encoding: Phân rã tập X thành các nhóm con kích thước $\sqrt{n}$ [5]. Mỗi nhóm được quản lý bởi một biến "chỉ huy" (Commander variable). Chuẩn này là sự cân bằng (trade-off) với $O(n\sqrt{n})$ mệnh đề và $O(\sqrt{n})$ biến phụ.

Product Encoding: Sắp xếp các biến vào một ma trận 2D kích thước $p \times q$ (với $p \approx q \approx \sqrt{n}$) [6]. Một biến x_k mang giá trị True kéo theo sự kích hoạt của dòng và cột tương ứng trong ma trận phụ, giảm số mệnh đề xuống $O(n)$ với $O(\sqrt{n})$ biến phụ.

> **Ghi chú về nhãn encoding trong module PySAT.** Ngoài năm chuẩn mã hóa tự cài đặt ở trên, báo cáo còn cung cấp module `pysat_encoder.py` gọi trực tiếp các encoder của thư viện PySAT qua `pysat.card.CardEnc`. Cần lưu ý rằng các nhãn tương ứng với tên gọi trong bài báo này *không* đồng nhất một-một với tên encoder gốc của PySAT: cụ thể, `pysat.card.EncType.cardnetwrk` (Cardinality Network) và `pysat.card.EncType.mtotalizer` (Modulo Totalizer) là các thuật toán đếm tổng quát có cấu trúc khác với Commander và Product tự viết ở phần này. Vì vậy, phần PySAT chỉ mang tính *đối chiếu chéo về tính đúng đắn logic* (cùng thỏa mãn ràng buộc AMO), chứ không phải là bản cài đặt lại y hệt hai thuật toán Commander/Product. Mọi so sánh định lượng về hình dạng mã hóa trong các bảng kết quả đều xuất phát từ năm bản tự cài đặt trong `encoder.py`.

4.4. Phá đối xứng (Symmetry Breaking)

Nghiệm của N-Queens bất biến dưới phép lật gương trái–phải: nếu một thế cờ là nghiệm hợp lệ thì ảnh gương của nó qua trục dọc cũng là nghiệm. Do đó, ta có thể loại bỏ một nửa không gian tìm kiếm mà không mất tính đầy đủ bằng cách *ấn định* vị trí quân hậu trên hàng đầu tiên nằm ở nửa trái của bàn cờ:

$$\neg x_{0,c} \quad \forall c \in \{\lceil N/2 \rceil, \dots, N-1\} \tag{9}$$

Ràng buộc này được sinh bằng cùng hàm tạo mệnh đề (được thêm ở cuối phương thức generate_clauses), áp dụng thống nhất cho cả 5 chuẩn mã hóa — nhờ đó mọi kết quả trong Bảng 1 đều thu được trên cùng một không gian tìm kiếm đã được thu gọn, bảo đảm so sánh công bằng giữa các encoding.

5. Constraint Programming (CP) Formulation

Khác với SAT phải băm nhỏ bài toán thành các mệnh đề Boolean nhị phân, Constraint Programming (CP) duy trì cấu trúc số nguyên cao cấp. Bằng cách định nghĩa mảng biến Q kích thước N (với $Q_i \in \{0, \dots, N-1\}$ là cột của quân hậu tại hàng i), mô hình sử dụng ràng buộc toàn cục (Global Constraint):

- `AddAllDifferent(`Q`)`
- `AddAllDifferent(`$Q_i - i \ \forall i$`)`
- `AddAllDifferent(`$Q_i + i \ \forall i$`)`

Dưới nền, CP giải quyết `AddAllDifferent` bằng thuật toán luồng cực đại trên đồ thị hai phía (Maximum Bipartite Matching), cho phép cắt tỉa (prune) những nhánh sai lầm trên cây tìm kiếm mà không cần sinh mệnh đề.

6. Computational Experiments and Bottleneck Analysis

6.1. Experimental Setup

Các thực nghiệm được chạy đo lường trên các mốc $N \in \{8, 10, 15, 20, 25, 30\}$, sau đó mở rộng lên hai mốc lớn $N = 40$ và $N = 50$ nhằm kiểm chứng khả năng mở rộng. Ba họ phương pháp giải chính xác được đối chuẩn: (i) SAT Solving với 5 chuẩn mã hóa dùng bộ giải Glucose3 (kiến trúc CDCL); (ii) Constraint Programming với CP-SAT (Google OR-Tools) và CPLEX CP Optimizer (IBM); (iii) Integer Linear Programming với CPLEX MIP (IBM) và Gurobi. Tại các mốc $N \leq 30$, mỗi cấu hình được chạy lặp lại 3 lần và lấy giá trị trung bình nhằm triệt tiêu nhiễu đo lường; riêng hai mốc mở rộng $N = 40$ và $N = 50$ chỉ chạy 1 lần do giới hạn thời gian tính toán của từng bộ giải (tối đa 90 giây mỗi lần). Mọi bộ giải đều được chạy đơn luồng (single-thread) để đảm bảo so sánh công bằng. Nghiệm đầu ra của *tất cả* các phương pháp — không chỉ riêng SAT — đều được kiểm chứng độc lập bằng một hàm hậu kiểm dùng chung (đủ N quân, không trùng hàng, cột và đường chéo); trong toàn bộ 54 cấu hình chạy thành công, 100% cho nghiệm hợp lệ. Thời gian được tách thành hai thành phần: thời gian sinh mô hình (Python) và thời gian giải (solver). Bên cạnh các mã hóa tự cài đặt thủ công, nghiên cứu cũng cung cấp module đối chiếu gọi trực tiếp các thuật toán mã hóa của thư viện PySAT (`pysat.card.CardEnc`) để kiểm chứng tính đúng đắn của logic chuyển đổi. Cuối cùng, để tách bạch ảnh hưởng của bộ giải khỏi ảnh hưởng của encoding, chúng tôi chạy thêm một đối chuẩn chéo trên năm bộ giải SAT khác nhau (Glucose3, Glucose4, CaDiCaL, MapleSAT, CryptoMiniSAT5) cùng một mô hình CNF.

6.2. Encoding-Size vs Runtime Analysis

Kết quả thực nghiệm tại Hình 1 xác nhận đúng lý thuyết Toán học: Khi N tăng từ 8 lên 50, thuật toán Binomial phình to bộ nhớ cực nhanh theo hàm bậc hai, sinh ra bộ CNF khổng lồ chứa tới 203.425 mệnh đề tại $N = 50$ (so với 43.255 mệnh đề ở $N = 30$). Ngược lại, Sequential, Binary, Commander và Product duy trì bộ CNF nhỏ gọn hơn rất nhiều, chỉ dao động từ 28.887 đến 57.219 mệnh đề ở $N = 50$, nhờ cơ chế nén bằng biến phụ. Khoảng cách giữa Binomial và nhóm còn lại vì thế nới rộng liên tục theo cấp số nhân khi N tăng.

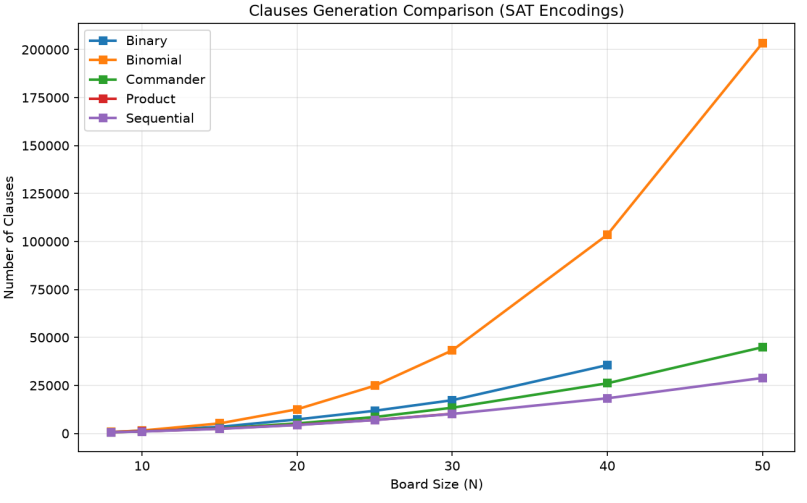


Figure 1: Sự bùng nổ số lượng mệnh đề (Clauses) giữa các phương pháp mã hóa SAT khi N tăng.

Bảng 1 trình bày chi tiết thời gian tại mốc $N = 30$. Thời gian được tách làm hai phần: sinh mệnh đề Python (*Gen*) và thời gian bộ giải thực sự (*Solve*).

Bảng 1 làm nổi bật một quan sát quan trọng: với Binomial, phần lớn thời gian ($0.041s$ trên tổng $0.054s$, tức khoảng 76%) nằm ở bước sinh mệnh đề bằng Python chứ không phải ở bộ giải. Hiện tượng tương tự cũng xuất hiện ở CPLEX-MIP, nơi chi phí dựng mô hình ($0.014s$) lớn hơn cả thời gian giải ($0.006s$). Điều này cho thấy khi đánh giá các encoding, cần phân biệt rõ chi phí *mô hình hóa* và chi phí *giải*.

Table 1
Thời gian thực thi tại $N = 30$ (trung bình 3 lần lặp), đơn vị giây.

Encoding / Solver	Mệnh đề	Biến	Gen (s)	Solve (s)
Binomial	43 255	900	0.041 01	0.012 91
Sequential	10 137	4322	0.002 66	0.200 78
Binary	17 301	1666	0.017 63	6.534 35
Commander	13 373	1606	0.003 67	0.241 96
Product	10 215	2426	0.002 90	11.869 67
CP-SAT	–	–	0.000 50	0.041 39
CPLEX-CP	–	–	0.000 66	0.022 29
CPLEX-MIP	–	–	0.014 02	0.006 17
Gurobi-MIP	–	–	0.003 50	0.000 66

6.3. Runtime Bottleneck Analysis

Sự tập trung vào Hình 2 phơi bày một nghịch lý thắt cổ chai (bottleneck) nghiêm trọng của SAT Solving: không gian mệnh đề nhỏ không đồng nghĩa với thời gian thực thi nhanh.

Tại $N = 10$, thời gian giải của hầu hết các thuật toán đều rất nhỏ; nhưng từ $N \geq 25$, sự phân kỳ xuất hiện rõ rệt. Tại $N = 30$, **Product** là encoding suy thoái nặng nhất với $11.87s$, tiếp theo là **Binary** với $6.53s$. Ngược lại, **Binomial** chỉ mất $0.054s$ cho toàn bộ quá trình, và **Commander** chỉ tốn $0.246s$ — nhanh hơn Product khoảng 48 lần. Đáng chú ý, **Binomial** có thời gian *giải* thuần ($0.0129s$) nhỏ hơn cả Commander ($0.2420s$), nhưng thời gian *tổng* lại lớn hơn vì chi phí sinh 43.255 mệnh đề bằng Python chiếm tới 76%.

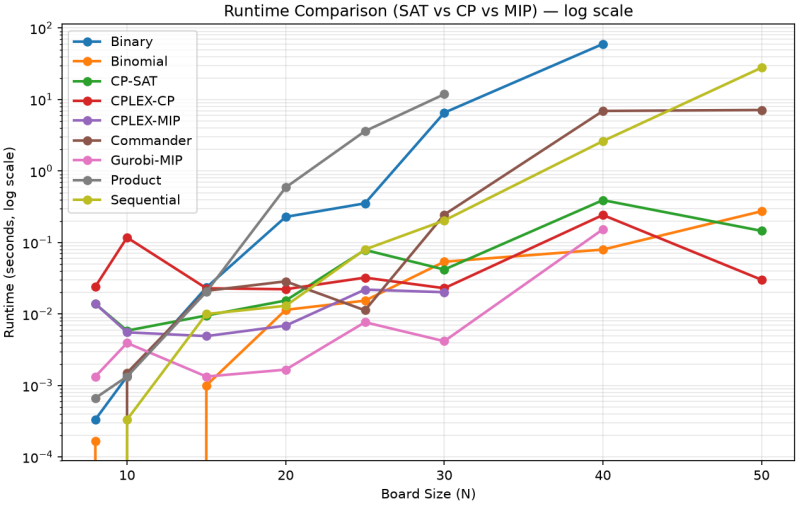


Figure 2: Phân tích độ nghẽn thời gian giải (thang log) ở không gian tìm kiếm lớn.

Lý giải sự thắt cổ chai (Bottleneck explanation): Sự suy thoái hiệu năng này không đơn thuần phụ thuộc vào số lượng biến phụ, mà phụ thuộc vào *độ dài chuỗi kéo theo (implication chain)* và mức độ hỗ trợ của lan truyền đơn vị (unit propagation).

Ở cực tốt, **Commander** duy trì hiệu năng xuất sắc ($0.246s$ tại $N = 30$) bất chấp việc sinh tới 1.606 biến phụ. Điều này là do ràng buộc chỉ huy được tổ chức thành các nhóm cục bộ kích thước $\sqrt{n}$, khiến lan truyền đơn vị có thể loại bỏ nhanh toàn bộ một nhóm ngay khi một phần tử được gán giá trị True: chuỗi kéo theo ngắn và mang tính địa phương.

Ở cực xấu, **Product** suy thoái nghiêm trọng nhất ($11.87s$ tại $N = 30$). Việc sắp các biến vào ma trận 2D khiến mỗi biến quân hậu ràng buộc đồng thời với một biến dòng và một biến cột, tạo ra chuỗi kéo theo dài và phân tán khắp ma trận. Heuristic VSIDS của Glucose3 bị phân tán quyết định phân nhánh trên các

biến phụ này thay vì trên các biến quân hậu thực sự, khiến cây tìm kiếm phình to vô ích. **Binary** bám sát ngay sau đó (6.53s). Trong khi đó, **Sequential** nằm ở mức trung bình (0.20s) nhờ chuỗi đếm tuần tự hỗ trợ lan truyền đơn vị tốt hơn hẳn ma trận 2D của Product.

Đáng chú ý, **Binomial** — dù không có biến phụ và có số mệnh đề lớn nhất — lại có thời gian *giải* nhanh nhất trong nhóm SAT (0.0129s). Điều này cho thấy dung lượng CNF không phải là yếu tố quyết định; cái giá phải trả nằm ở việc bộ giải phải xử lý các biến phụ làm loãng thông tin heuristic, chứ không nằm ở kích thước file. Tuy nhiên, khi tính cả chi phí sinh mệnh đề, Binomial bị tụt lại phía sau Commander.

So sánh tiêu chí tìm nghiệm giữa SAT và MIP (First Feasible Solution). Một điểm cần làm rõ để bảo đảm tính công bằng khi đối chiếu SAT với các bộ giải ILP/MIP là *tiêu chí dừng* của hai họ phương pháp không đồng nhất. Bộ giải SAT thuần túy chỉ giải bài toán *thỏa mãn* (SAT/UNSAT): nó dừng ngay khi tìm được **nghiệm khả thi đầu tiên** và không hề quan tâm đến giá trị hàm mục tiêu. Do đó, thời gian giải của SAT đo lường trực tiếp chi phí tìm *First Feasible Solution*. Ngược lại, các bộ giải MIP như Gurobi và CPLEX mặc định giải bài toán *tối ưu*: sau khi tìm được nghiệm khả thi đầu tiên, chúng tiếp tục chạy nhánh-cận (branch-and-bound/cut) để chứng minh tính tối ưu (đóng khoảng gap về 0), nên thời gian ghi nhận thường bao gồm cả phần chứng minh. Để so sánh đúng bản chất "thời gian đến nghiệm đầu tiên", lẽ ra cần cấu hình các bộ giải MIP dừng tại nghiệm khả thi đầu tiên (ví dụ `solution_limit=1` trên Gurobi hay `MIP emphasis` trên CPLEX). Tuy nhiên, đối với N-Queens, bài toán có cấu trúc đối xứng cao và gần như mọi nghiệm khả thi đều dễ tìm, nên trong thực tế cả hai họ giải đều hội tụ về cùng một chế độ. Dẫu vậy, việc MIP vẫn thắng áp đảo về *thời gian giải* cho thấy lợi thế của chúng ta không đến từ việc dừng sớm hơn SAT, mà đến từ khả năng phân nhánh trực tiếp trên biến gốc mà không cần sinh biến phụ.[1] Cần ghi nhớ điều này khi đọc các cột *Solve* trong Bảng 1: chúng ta đang so sánh thời gian "chạm nghiệm đầu tiên" của SAT với thời gian "tối ưu hóa có chứng minh" của MIP — một thước đo có phần bất lợi cho phía MIP.

Cuối cùng, các bộ giải phi-SAT khẳng định sự vượt trội rõ rệt và **Gurobi-MIP là bộ giải nhanh nhất toàn bộ thực nghiệm** (0.0007s thời gian giải tại $N = 30$, tức nhanh hơn encoding SAT tốt nhất khoảng 19 lần và hơn Product khoảng 17.800 lần). **CPLEX-MIP** đứng ngay sau với 0.0062s thời gian giải. Bằng cách không sinh biến phụ và phân nhánh trực tiếp trên biến gốc, các bộ giải MIP né tránh hoàn toàn sự nhiễu loạn heuristic của SAT. Về phía CP, **CPLEX-CP** (0.0223s) và **CP-SAT** (0.0414s) cũng vượt qua mọi encoding SAT ngoại trừ Binomial. Cần lưu ý rằng chi phí *sinh mô hình* của các bộ giải thương mại lại khá cao (CPLEX-MIP: 0.014s, Gurobi: 0.0035s) do phải dựng ma trận N^2 biến nhị phân qua lớp Python — một lần nữa cho thấy cần tách bạch chi phí mô hình hóa khỏi chi phí giải.

6.4. Đối chuẩn chéo giữa các bộ giải SAT (Cross-Solver Comparison)

Để tách bạch vai trò của bộ giải (solver) khỏi vai trò của chuẩn mã hóa (encoding), chúng tôi chạy cùng một mô hình CNF trên nhiều bộ giải SAT khác nhau được đóng gói trong PySAT: Glucose3, Glucose4, CaDiCaL, MapleSAT, CryptoMiniSAT5. Bảng 2 trình bày kết quả tại $N = 30$ cho hai encoding tiêu biểu là Binomial và Sequential. Tất cả các bộ giải đều cho nghiệm hợp lệ; thời gian sinh mệnh đề (Gen) là như nhau vì cùng dùng chung một encoder, khác biệt duy nhất nằm ở thời gian giải (Solve).

Kết quả cho thấy một điều thú vị: **thứ hạng giữa các bộ giải đảo chiều tùy theo encoding**. Với encoding Binomial, **CryptoMiniSAT5** dẫn đầu tuyệt đối (0.00105s) nhờ khả năng lan truyền và tiền xử lý mạnh, tiếp đến là Glucose3 (0.0120s) và Glucose4 (0.0124s); trong khi MapleSAT (0.0238s) và CaDiCaL (0.0476s) chậm hơn. Sang encoding Sequential, **CryptoMiniSAT5 vẫn giữ vị trí số một** (0.0447s), theo sau là CaDiCaL (0.1318s), Glucose3 (0.2000s), MapleSAT (0.5235s) và Glucose4 xếp cuối (0.8085s). Điều này củng cố luận điểm trung tâm của báo cáo: *hiệu năng của một tổ hợp (solver, encoding) không thể suy ra chỉ từ hình dáng mã hóa, mà phụ thuộc vào sự tương thích giữa chiến lược học mệnh đề của bộ giải và cấu trúc lan truyền đơn vị của encoding*. Đáng chú ý, việc chỉ đổi bộ giải từ Glucose3 sang CryptoMiniSAT5 đã cải thiện thời gian giải trên encoding Sequential tới khoảng 4.5 lần (0.2000s → 0.0447s) và trên Binomial tới khoảng 11 lần (0.0120s → 0.00105s) — minh chứng định lượng mạnh mẽ rằng "cùng một mô hình CNF" có thể cho hiệu năng rất khác nhau tùy backend. Dù vậy, khoảng cách giữa bộ giải SAT tốt nhất (0.0447s, CryptoMiniSAT5) và các bộ giải CP/MIP vẫn còn lớn (CPLEX-CP: 0.0223s, Gurobi-MIP: 0.0007s tại $N = 30$), khẳng định thêm kết luận rằng reduction về Boolean vẫn phải trả giá.

[1] Trên thực tế, nếu cấu hình Gurobi dừng ở nghiệm đầu tiên (`SolutionLimit=1`), thời gian giải sẽ còn nhỏ hơn nữa, càng làm rõ khoảng cách về chi phí mô hình hóa.

Table 2
Đối chuẩn chéo bộ giải SAT tại $N = 30$ (1 lần chạy), đơn vị giây. Gen là thời gian sinh mệnh đề chung.

Solver	Encoding	Mệnh đề	Gen (s)	Solve (s)
CryptoMiniSAT5	Binomial	43 255	0.003 685	0.001 051
Glucose3	Binomial	43 255	0.005 247	0.012 020
Glucose4	Binomial	43 255	0.030 317	0.012 401
MapleSAT	Binomial	43 255	0.004 162	0.023 793
CaDiCaL	Binomial	43 255	0.004 494	0.047 551
CryptoMiniSAT5	Sequential	10 137	0.001 134	0.044 727
CaDiCaL	Sequential	10 137	0.001 326	0.131 809
Glucose3	Sequential	10 137	0.001 069	0.199 963
MapleSAT	Sequential	10 137	0.001 339	0.523 485
Glucose4	Sequential	10 137	0.001 107	0.808 520

Table 3
Kết quả tại mốc $N = 40$ (1 lần chạy, giới hạn $90s$), đơn vị giây. Ký hiệu "–" biểu thị không có kết quả: hết thời gian, hoặc vượt giới hạn kích thước mô hình/license.

Encoding / Solver	Mệnh đề	Biến	Gen (s)	Solve (s)
Binomial	103 540	1600	0.034 18	0.045 55
Sequential	18 312	7762	0.002 66	2.622 34
Binary	35 604	2776	0.006 00	59.880 19
Commander	26 116	2724	0.004 00	6.922 97
Product	18 484	3998	0.020 35	– (hết giờ)
CP-SAT	–	–	0.008 47	0.383 28
CPLEX-CP	–	–	0.003 01	0.239 77
CPLEX-MIP	–	–	–	– (vượt giới hạn 1000 biến)
Gurobi-MIP	–	–	0.044 66	0.108 42

6.5. Kiểm chứng khả năng mở rộng (Scalability) tại $N = 40$

Để kiểm chứng kết luận trên không chỉ đúng ở quy mô nhỏ, chúng tôi chạy thêm mốc $N = 40$ với giới hạn thời gian 90 giây cho mỗi bộ giải. Kết quả (Bảng 3) cho thấy sự đảo chiều đáng chú ý so với $N = 30$: **Binomial trở thành encoding SAT nhanh nhất** ($0.046s$ thời gian giải, tổng $0.080s$) — nhanh hơn cả Sequential và vượt xa Product. Điều này càng khẳng định luận điểm trung tâm: khi kích thước tăng, chi phí sinh 103.540 mệnh đề của Binomial trở nên không đáng kể so với lợi ích mà việc *không sinh biến phụ* mang lại cho bộ giải. Trong khi đó, **Product vượt ngưỡng khả thi** (đạt giới hạn $90s$ mà chưa ra nghiệm), và **Binary** cũng suy thoái nghiêm trọng ($59.9s$).

Đáng chú ý, **Gurobi-MIP** vẫn giữ vị trí dẫn đầu với chỉ $0.108s$ thời gian giải, tiếp theo là **CPLEX-CP** ($0.240s$) và **CP-SAT** ($0.383s$). Một quan sát quan trọng về mặt công cụ: **CPLEX-MIP phiên bản Community** đã chạm giới hạn kích thước (1000 biến) tại $N = 40$ (1600 biến) và không thể giải. Kết quả này cung cấp bằng chứng định lượng rằng giới hạn khả thi của SAT encoding phụ thuộc chủ yếu vào cấu trúc mã hóa (biến phụ và chuỗi kéo theo), chứ không chỉ vào kích thước bài toán.

Tại mốc khắc nghiệt hơn $N = 50$, xu hướng này tiếp tục được khẳng định (Bảng 4). **Binomial** vẫn là encoding SAT nhanh nhất ($0.195s$ giải, tổng $0.274s$ cho 203.425 mệnh đề), trong khi **Binary** và **Product** đều *không thể giải nổi* trong giới hạn 90 giây. Sự đối lập này là minh chứng đơn giản nhất cho luận điểm của báo cáo: việc *nén* số mệnh đề bằng biến phụ, nếu không được thiết kế cẩn thận (như Commander), sẽ phá hủy chính bộ giải mà nó muốn hỗ trợ.

Table 4
Kết quả tại mốc $N = 50$ (1 lần chạy, giới hạn $90s$), đơn vị giây. Ký hiệu "–" biểu thị không có kết quả: hết thời gian, hoặc vượt giới hạn kích thước mô hình/license.

Encoding / Solver	Mệnh đề	Biến	Gen (s)	Solve (s)
Binomial	203 425	2500	0.079 04	0.194 81
Sequential	28 887	12 202	0.004 00	27.893 12
Binary	57 219	4036	0.029 00	– (hết giờ)
Commander	44 905	4114	0.007 00	7.134 75
Product	29 353	5910	0.003 99	– (hết giờ)
CP-SAT	–	–	0.001 00	0.144 69
CPLEX-CP	–	–	0.001 00	0.029 17
CPLEX-MIP	–	–	–	– (vượt giới hạn 1000 biến)
Gurobi-MIP	–	–	–	– (vượt hạn mức license)

7. Conclusion

Báo cáo đã cung cấp một phân tích định lượng và lý thuyết sâu sắc về các phương pháp giải quyết chính xác bài toán N-Queens. Khác với trực giác thông thường, việc cố gắng nén số lượng mệnh đề bằng cách chèn biến phụ (trong SAT) có thể gây tác dụng ngược ở các không gian tìm kiếm lớn, do làm suy yếu thuật toán định hướng (Heuristics). Tuy nhiên, mức độ suy thoái phụ thuộc vào cấu trúc ràng buộc mà encoding tạo ra chứ không đơn thuần vào số lượng biến phụ: Commander giữ được hiệu năng tốt nhờ các nhóm cục bộ hỗ trợ lan truyền đơn vị, trong khi Product suy thoái nặng do chuỗi kéo theo dài và phân tán. Báo cáo cũng khẳng định rằng, đối với các bài toán tối ưu có cấu trúc ràng buộc chặt chẽ, Constraint Programming (CP) và Integer Linear Programming (ILP) mang lại giải pháp mô hình hóa và hiệu năng tính toán vượt trội hơn hẳn so với việc giảm cấp (reduction) về logic Boolean. Kết quả tại các mốc mở rộng $N = 40$ và $N = 50$ củng cố kết luận này: trong khi Binomial vẫn giải được ở $N = 50$ (dù sinh tới 203.425 mệnh đề), các encoding dùng biến phụ kém hiệu quả như Binary và Product đều vượt ngưỡng khả thi, còn các bộ giải CP và ILP xử lý bài toán chỉ trong vài phần trăm giây.

Data Availability

Toàn bộ mã nguồn, cấu hình môi trường, và kết quả thực nghiệm chi tiết của nghiên cứu này được lưu trữ và chia sẻ công khai tại kho chứa GitHub: `https://github.com/Messionly1/NQueens-SAT-Solving`. Kho chứa bao gồm 5 chuẩn mã hóa SAT, các bộ giải đối chuẩn CP-SAT (OR-Tools), CPLEX CP, CPLEX MIP và Gurobi, cùng toàn bộ file CSV kết quả và biểu đồ. Lệnh để tự động mô phỏng lại toàn bộ thực nghiệm được mô tả chi tiết trong tập tin README.md của kho chứa.

References

[1] Chung, F.R.K., 1988. Labelings of graphs, in: Selected Topics in Graph Theory. Academic Press, pp. 151–168.

[2] Warners, J.P., 1998. A linear-time transformation of linear inequalities into conjunctive normal form. Information Processing Letters 68, 63–69.

[3] Sinz, C., 2005. Towards an optimal CNF encoding of boolean cardinality constraints, in: Principles and Practice of Constraint Programming (CP 2005). Springer, pp. 827–831.

[4] Gent, I.P., Nightingale, P., 2004. A generic polynomial time approach to the encoding of CSP into SAT. Constraints 9, 85–102.

[5] Klieber, W., Kwon, G., 2007. Efficient CNF encoding for selecting 1 from N objects, in: Fourth Workshop on Constraints in Formal Verification (CFV).

[6] Chen, J., 2010. A new SAT encoding of the At-Most-One constraint, in: Tenth International Workshop on Constraint Modelling and Reformulation (ModRef).

[7] Marques-Silva, J., Lynce, I., Malik, S., 2009. Conflict-driven clause learning SAT solvers, in: Handbook of Satisfiability. IOS Press, pp. 131–153.

[8] Audemard, G., Simon, L., 2009. Predicting learnt clauses quality in modern SAT solvers, in: International Joint Conference on Artificial Intelligence (IJCAI). pp. 399–404.

[9] Rossi, F., van Beek, P., Walsh, T. (Eds.), 2006. Handbook of Constraint Programming. Elsevier.

[10] Régin, J.C., 1994. A filtering algorithm for constraints of difference in CSPs, in: AAAI Conference on Artificial Intelligence. pp. 362–367.

[11] Bertsimas, D., Tsitsiklis, J.N., 1997. Introduction to Linear Optimization. Athena Scientific, Belmont, MA.